MIT 8103 Advanced Database Systems

Banking and Fintech Transaction Processing Platform · Abdul Usman · Student ID 301919351

MIT 8103 Advanced Database Systems

Banking and Fintech Transaction Processing Platform · Abdul Usman · Student ID 301919351

Case study

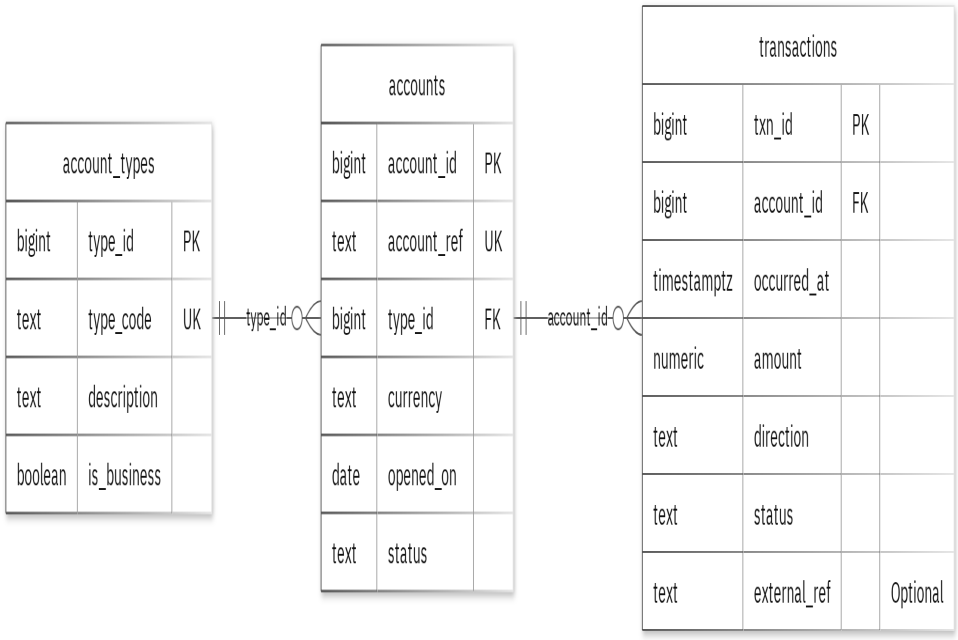
My case study is a Banking and Fintech Transaction Processing Platform, that is designed to help operations analysts to review account activities and platform engineers to maintainthe accuracy and integrity of financial transactions.

The database consists of three main tables: account_types, accounts and transactions. It supports recording transactions against customer accounts, retrieving account transaction history within specified date ranges, and summarising transaction amounts by account and direction (credit or debit).

Portfolio 1: Database Design and Modelling

I designed the relational database using three tables, with primary keys to identify records and foreign keys to link the tables. I also added constraints to ensure that the database follows the required business rules. For example, each transaction must have a unique external reference to prevent transaction duplicate.

I also demonstrated how the sample data meets the requirements of 1NF, 2NF and 3NF. To test the database, I attempted 26 invalid inserts, and all were rejected without affecting the existing valid records. This helped me confirm that the constraints were working correctly and that the database was protecting data integrity.



Banking and Fintech Transaction Processing Platform ER diagram

Figure 1. Tables, keys and relationships. PK: primary key; FK: foreign key; UK: unique value.

Portfolio 2: Query Processing and Optimisation

I ran three SQL queries using the sample data. The JOIN query returned one record, showing SYN-ACCOUNT-001 with the PERSONAL account type.

To test query performance, I used a separate dataset containing 50,000 records. Before creating the index, the average lookup time was 2.454 ms. After indexing, it dropped to 0.322 ms. Both tests returned the same records, and the execution plan confirmed that the new index was being used.

This shows that indexing improved the lookup speed in my test. However, the results are specific to this query and the environment in which the test was performed.

Portfolio 3: Transactions and Concurrency

I used a temporary counter in the transactions table to test how COMMIT, ROLLBACK and READ COMMITTED isolation work. I observed that changes made before a COMMIT were not visible to the other session, while using ROLLBACK restored the previous saved value.

I also tested what happens when two sessions update the counter at the same time. Without locking, both sessions attempted to increment it, but the final value was 2 instead of the expected 3. After using SELECT FOR UPDATE, Session B had to wait for Session A to complete before making its update. This time, the final value was 3. The temporary counter was used only for this exercise and did not change the original business fields in the case study.

Portfolio 4: NoSQL and Advanced Data Models

I used the transaction_events collection in MongoDB to store event documents for the case study. I added two sample events and ran four queries, all of which returned the expected results. I also tested the validation rules by attempting to insert an invalid date type and a duplicate event ID. MongoDB rejected both entries as expected.

The embedded snapshots allow related information to be stored alongside each event, while the details field can contain different information depending on the type of event. All the events used were fictional, and PostgreSQL remains responsible for enforcing the relational database rules.

Repository and evidence

https://github.com/Habj93/Usman_Abdul_301919351

The portfolio folders contain the SQL and JavaScript, explanations, query plans, result tables and screenshots.

Portfolio 5: Distributed and Cloud Database Exercise

I proposed using three MongoDB replica members, with each member hosted in a separate failure zone to reduce the risk of downtime. I explained how primary elections work, why majority acknowledgement is important, and how delays in secondary replication can affect the system. I also discussed the advantages and trade-offs of using sharding for the transaction_events collection.

The discussion shows that replication helps keep the database available, while backups are needed to recover lost or damaged data. PostgreSQL would also require a separate availability and recovery plan. This was completed as a discussion-based exercise, as permitted by the assessment. I did not deploy a live replica set or carry out an actual failover test.

Individual reflection

I kept account_types, accounts and transactions in separate tables to avoid repeating data. I also added constraints to prevent duplicate transaction references and ensure that required values and record relationships are valid.

The two-session test showed how concurrent updates can cause one update to be lost. Using row locking ensured both increments were applied correctly. The index test also showed that queries can return the same results but require different amounts of database work, which is why checking the execution plan matters.

AI-use declaration

I used GitHub Copilot and the portfolio generator to help create the database model, scripts and explanations. The generator also ran the PostgreSQL and MongoDB exercises, checked the results and captured screenshots. However, I am responsible for reviewing the work, confirming that the results are correct and being able to explain what I submitted.

References

- PostgreSQL: constraints
- PostgreSQL: EXPLAIN
- PostgreSQL: isolation
- PostgreSQL: row locks
- MongoDB: data modelling
- MongoDB: replication
- MongoDB: sharding
- Mermaid: ER diagrams